

Lab 1: NumPy, NLTK, and Files

Arrays, stemming versus lemmatization, and a movie script.

Work in a **Jupyter** notebook. Number the exercises. Do notes **1.1** through **1.3** first. You will need `numpy` and `nltk`.

Download [nemo.txt](#) into the same folder as the notebook (or use a relative path).

When you are done: **File** → **Download as** → **HTML**, then upload the HTML on Canvas. Save the notebook before you export.

```
import numpy as np
import nltk
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

nltk.download("punkt")
nltk.download("wordnet")
```

If `word_tokenize` asks for `punkt_tab`, run `nltk.download("punkt_tab")` as well.

1. NumPy

1. Create a NumPy array `A` with shape `(6, 6)` using `np.random.rand()`.
2. Print the entire third row (index `2`).
3. Print every odd column (0-based: columns 0, 2, 4, ...).
4. Build a dictionary with keys `"even"` and `"odd"`. The value for `"even"` is the sum of entries whose row-plus-column index `i + j` is even. Same idea for `"odd"`.
5. A class is a blueprint for objects. Write a class `matrixMaker` with: - attributes `m` and `n` - a matrix `A` of shape `(m, n)` with entries from `(N(0, 1))` - a method `specialSum()` that returns the sum of entries of `A` that are greater than `0.5`

Create `test = matrixMaker(5, 5)` and call `test.specialSum()`.

2. NLTK

```
plurals = [
    "caresses", "flies", "dies", "mules", "denied",
    "died", "agreed", "owned", "humbled", "sized",
    "meeting", "stating", "siezing", "itemization",
    "sensational", "traditional", "reference", "colonizer",
    "plotted",
]
```

1. Import NLTK. Use the `plurals` list above.
2. Stem every word with a loop and `PorterStemmer().stem()`.
3. Lemmatize every word with `WordNetLemmatizer`.
4. Tokenize this sentence with `word_tokenize` and print the tokens:

```
Wow, we can finally stop using the split function!
```

Stemming chops a suffix and may leave a non-word (`flying` → `fli`). Lemmatization aims at a dictionary form and uses WordNet. Tokenization splits text into words and punctuation. `str.split()` does not treat punctuation as its own token.

3. Files

Required reading: [Reading and writing files in Python](#).

1. Open `nemo.txt`.
2. How many words are in the script? (`split()` on each line is enough.)
3. How many times did a character speak? Who spoke the most?

Speaker names in this file sit alone on a line, in ALL CAPS, with no `?`, `.`, or `!`. Count those lines. Sort the counts.